

PREPARE Repo

PREPARE: Data Collection & Data Integrity

The goal of the preparation session is to set up my SQL environment and aggregate the monthly data to define a quarter-one dataframe. I downloaded all the data with Q1 2025 and manufactured a CSV file for the aggregated data using UNION ALL.

Query Log:

- Query 1: Preview data
- Query 2: Count individual tables
- Query 3: Count individual tables with an alias
- Query 4: Create SSOT
- Query 5: Verify SSOT Count
- Query 6: Preview the combined dataset.
- Query 7: Establish baseline count.
- Query 8a: Count member rides.
- Query 8b: Count casual rides.
- Query 9a: Check for missing user type.
- Query 9b: Check for missing start times.
- Query 9c: Check for missing end times.
- Query 9d: Check for missing start station.
- Query 9e: Check for missing end station.
- Query 10: Preview rides with complete station data.

Query 1: Preview data

None

SQL

```
SELECT *  
FROM `cyclistic-case-study-493121.CycleData.jan_2025`  
LIMIT 10;
```

Result/Observation: Verifying that the data in properly loaded into BigQuery.

Query 2: Count individual tables

None

SQL

```
-- Query 2a, 2b, 2c
```

```

-- January
-- Verify expected 138,689 rows before merging datasets

SELECT COUNT(*)
FROM `cyclistic-case-study-493121.CycleData.jan_2025`;

-- Expected 138,689

-- February
-- Verify expected 151,880 rows before merging datasets

SELECT COUNT(*)
FROM `cyclistic-case-study-493121.CycleData.feb_2025`;

-- Expected 151,880

-- March
-- Verify expected 298,155 rows before merging datasets

SELECT COUNT(*)
FROM `cyclistic-case-study-493121.CycleData.mar_2025`;

-- Expected 298,155

```

Result/Observation: Validating counts to ensure that all data is accounted. Everything matches.

Query 3: Count individual tables with alias

```

None
SQL
-- Query 3a, 3b, 3c

-- January
-- Verify expected 138,689 rows before merging datasets

SELECT COUNT(*) AS total_rows

```

```

FROM `cyclistic-case-study-493121.CycleData.jan_2025`;

-- Expected 138,689

-- Feburary
-- Verify expected 151,880 rows before merging datasets

SELECT COUNT(*) AS total_rows
FROM `cyclistic-case-study-493121.CycleData.feb_2025`;

-- Expected 151,880

-- March
-- Verify expected 298,155 rows before merging datasets

SELECT COUNT(*) AS total_rows
FROM `cyclistic-case-study-493121.CycleData.mar_2025`;

-- Expected 298,155

```

Result/Observation: ensuring that all subsets of data has been properly loaded.

Query 4: Create SSOT

None

SQL

-- Query 4

```

CREATE TABLE `cyclistic-case-study-493121.CycleData.q1_2025` as
SELECT 'January' as Month, * FROM
`cyclistic-case-study-493121.CycleData.jan_2025`
UNION ALL
SELECT 'Feburary' as Month,* FROM
`cyclistic-case-study-493121.CycleData.feb_2025`
UNION ALL
SELECT 'March' as Month, * FROM
`cyclistic-case-study-493121.CycleData.mar_2025`;

```

Result/Observation: Created an aggregated table, including all months to determine all data for the first quarter of the year.

Query 5: Verify SSOT count

None

SQL

-- Query 5

```
SELECT COUNT(*) as Total_Rows  
FROM `cyclistic-case-study-493121.CycleData.q1_2025`;
```

-- Expected: 588724

Result/Observation:

Validating that all the data was properly combined, and no data was corrupted during the aggregation process.

Query 6: Preview combined dataset

None

SQL

```
SELECT *  
FROM `cyclistic-case-study-493121.CycleData.q1_2025`  
LIMIT 10;
```

Result/Observation:

The format of the data looks accurate. I notice that we have all the needed columns, and the data structure is accurate.

Query 7: Establish baseline count

None

SQL

```
SELECT COUNT(*) AS COUNT
FROM `cyclistic-case-study-493121.CycleData.q1_2025`;
```

Result/Observation:

The count is 588724, which is what is expected. I was able to retrieve the correct number of values.

Query 8a: Count member rides

None

SQL

```
SELECT COUNT(*) AS member_rides
FROM `cyclistic-case-study-493121.CycleData.q1_2025`
WHERE member_casual = 'member';
-- Expected: ~450k
```

Result/Observation:

I identified the correct output for member observation, ensuring integrity in the member group.

Query 8b: Count casual rides

None

SQL

```
SELECT COUNT(*) AS casual_rides
FROM `cyclistic-case-study-493121.CycleData.q1_2025`
WHERE member_casual = 'casual';
-- Expected: ~138K
```

Result/Observation:

Retrieved the correct number of casual users, identifying that my data is accurate and there was no corruption during transformation.

Query 9a: Check for missing user type

None

SQL

```
SELECT COUNT(*) AS missing_member_casual
FROM `cyclistic-case-study-493121.CycleData.q1_2025`
WHERE member_casual IS NULL;
-- Expected: 0
```

Result/Observation:

Identified non-missing values in the member_casual category, identify that all observations align with categorization requirements.

Query 9b: Check for missing start times

None

SQL

```
SELECT COUNT(*) AS missing_started_at
FROM `cyclistic-case-study-493121.CycleData.q1_2025`
WHERE started_at IS NULL;
-- Expected: 0
```

Result/Observation:

There are no null values for started_at, identifying that all observations had an associated start-time.

Query 9c: Check for missing end times

None

SQL

```
SELECT COUNT(*) AS missing_ended_at
FROM `cyclistic-case-study-493121.CycleData.q1_2025`
WHERE ended_at IS NULL;
-- Expected: 0
```

Result/Observation:

Identified no missing ended_at values, identifying no missing finish time in the dataset.

Query 9d: Check for missing start station

None

SQL

```
SELECT COUNT(*) AS missing_start_station
FROM `cyclistic-case-study-493121.CycleData.q1_2025`
WHERE start_station_name IS NULL;
```

Result/Observation:

There are null values, identifying that a larger proportion of the dataset is missing start stations.

Query 9e: Check for missing end station

None

SQL

```
SELECT COUNT(*) AS missing_end_station
FROM `cyclistic-case-study-493121.CycleData.q1_2025`
WHERE end_station_name IS NULL;
```

Result/Observation:

Identified null values, signifying that a large portion of this data does not have an associated end station.

Query 10: Preview rides with complete station data

None

SQL

```
SELECT COUNT(*) AS rides_with_stations
FROM `cyclistic-case-study-493121.CycleData.q1_2025`
WHERE start_station_name IS NOT NULL
      AND end_station_name IS NOT NULL;
-- Expected: ~417,000
```

Result/Observation:

Identified that there is still a significant amount of data that has been collected at start stations and end stations.

PROCESS Repo

PROCESS: Data Cleaning & Transformation

The goal of the cleaning and transformation stage to convert the newly curated Q1 data into quality data that is representative and usable within analysis. The final project on this stage is a dataset with little to no unrepresentative data, meaning the data should reflect the real world. Also separation of time data between hours, days, and weeks is needed to further analyse time trends.

Query Log:

- Query 11: Identify rides under 1 minute.
- Query 12: Identify rides over 24 hours.
- Query 13: Test combined filtering with AND.
- Query 14: Alternative filtering with BETWEEN.
- Query 15: Final combined preview, duration + station completeness.
- Query 16: Create cleaned table with duration + station filters.
- Query 17: Verify cleaned table row count
- Query 18: Preview cleaned dataset with specific columns
- Query 19: Calculate ride duration (preview)
- Query 20: Extract day of week (preview)
- Query 21: Extract hour of day (preview)
- Query 22: Categorize trip type with CASE (preview)
- Query 23: Preview combined features
- Query 24: Create processed dataset (all features)

Query 11: Identify rides under 1 minute

None

SQL

```
SELECT *
FROM `cyclistic-case-study-493121.CycleData.q1_2025`
WHERE TIMESTAMP_DIFF(ended_at,started_at,MINUTE) < 1
LIMIT 100;
```

Result/Observation:

Identified rides that are too short to be logically accurate.

Query 12: Identify rides over 24 hours

None

SQL

```
SELECT *
FROM `cyclistic-case-study-493121.CycleData.q1_2025`
WHERE TIMESTAMP_DIFF(ended_at,started_at,MINUTE) > 1440
LIMIT 100;
```

Result/Observation:

Identified observations that had time over an amount of time that was too long.

Query 13: Test combined filtering with AND

None

SQL

```
SELECT COUNT(*) AS valid_ride_count
FROM `cyclistic-case-study-493121.CycleData.q1_2025`
WHERE TIMESTAMP_DIFF(ended_at, started_at, MINUTE) >= 1
      AND TIMESTAMP_DIFF(ended_at,started_at,MINUTE) > 1440;
```

Result/Observation:

There are 576200 observations, identifying which observations are useful to provide an accurate and representative view of reality.

Query 14: Alternative filtering with BETWEEN

None

SQL

```
SELECT COUNT(*) AS ride_count_alt
FROM `cyclistic-case-study-493121.CycleData.q1_2025`
WHERE TIMESTAMP_DIFF(ended_at, started_at, MINUTE) BETWEEN 1 and 1440;
```

Result/Observation:

Identifying the number of valid entries in the dataset to provide a cleaned version for analysis.

Query 15: Final combined preview, duration + station completeness

None

SQL

```
SELECT COUNT(*) AS valid_ride_count
FROM `cyclistic-case-study-493121.CycleData.q1_2025`
WHERE TIMESTAMP_DIFF(ended_at, started_at, MINUTE) BETWEEN 1 AND 1440
      AND start_station_name IS NOT NULL
      AND end_station_name IS NOT NULL;
-- Expected+ ~417,000
```

Result/Observation:

Fully removing any data that would misrepresent the analysis and invalidate the results. In this context, we want the duration of rides to be accurate and not include outliers (rides less than 1 minute or more than 1440 minutes [24 hours]) and entries that have an accurate starting/ending point.

Query 16: Create a cleaned table with duration + station filters

None

SQL

```
CREATE TABLE `cyclistic-case-study-493121.CycleData.q1_2025_cleaned` AS
SELECT *
FROM `cyclistic-case-study-493121.CycleData.q1_2025`
WHERE TIMESTAMP_DIFF(ended_at, started_at, MINUTE) BETWEEN 1 AND 1440
      AND start_station_name IS NOT NULL
      AND end_station_name IS NOT NULL;
```

Result/Observation:

Fully integrated cleaning methodology into a finalized dataset, prepared for analysis.

Query 17: Verify cleaned table row count

None

SQL

```
SELECT COUNT(*) AS total_rows
FROM `cyclistic-case-study-493121.CycleData.q1_2025_cleaned`;
```

Result/Observation:

Validating the newly created dataset to ensure creation is accurate and as expected.

Query 18: Preview cleaned dataset with specific columns

None

SQL

```
SELECT
    ride_id,
    started_at,
    ended_at
    start_station_name,
    end_station_name,
    member_casual
FROM `cyclistic-case-study-493121.CycleData.q1_2025_cleaned`
LIMIT 10;
```

Result/Observation:

Only selects the needed columns to prevent spending on unnecessary queries.

Query 19: Calculate ride duration (preview)

None

SQL

```
SELECT
    ride_id,
    started_at,
    ended_at,
    TIMESTAMP_DIFF(ended_at, started_at,
    MINUTE) AS ride_length_minutes
FROM `cyclistic-case-study-493121.CycleData.q1_2025_cleaned`
```

```
LIMIT 10;
```

Result/Observation:

Generated the duration in rides to support duration analysis.

Query 20: Extract day of week (preview)

None

SQL

```
SELECT
    ride_id,
    started_at,
    EXTRACT(DAYOFWEEK FROM started_at) AS day_number
FROM `cyclistic-case-study-493121.CycleData.q1_2025_cleaned`
LIMIT 10;
```

Result/Observation:

Provides the day of the week to help support analysis of daily trends.

Query 21: Extract hour of day (preview)

None

SQL

```
SELECT
    ride_id,
    started_at,
    EXTRACT(HOUR FROM started_at) AS hour_number
FROM `cyclistic-case-study-493121.CycleData.q1_2025_cleaned`
LIMIT 10;
```

Result/Observation:

Extracted all hours of the day so the data can account for time of day to determine daily trends.

Query 22: Categorize trip type with CASE (preview)

None

SQL

```
SELECT
    ride_id,
    start_station_name,
    end_station_name,
    CASE
        WHEN start_station_name = end_station_name THEN 'Round Trip'
        ELSE 'One Way'
    END as trip_type
FROM `cyclistic-case-study-493121.CycleData.q1_2025_cleaned`
LIMIT 10;
```

Result/Observation:

Provides a new category to see how many people use the bike to and from their location, and how many use it to get to a location only.

Query 23: Preview combined features

None

SQL

```
SELECT
    ride_id,
    member_casual,
    TIMESTAMP_DIFF(ended_at, started_at, MINUTE) AS
ride_length_minutes,
    EXTRACT(DAYOFWEEK FROM started_at) AS day_number,
    EXTRACT(HOUR FROM started_at) AS hour_number,
    CASE
        WHEN start_station_name = end_station_name THEN 'Round Trip'
        ELSE 'One Way'
    END as trip_type
FROM `cyclistic-case-study-493121.CycleData.q1_2025_cleaned`
```

```
LIMIT 10;
```

Result/Observation: combining all engineered features to display what the new engineered data would look like.

Query 24: Create processed dataset (all features)

None

SQL

```
CREATE OR REPLACE TABLE
`cyclistic-case-study-493121.CycleData.q1_2025_processed`

SELECT *,
    TIMESTAMP_DIFF(ended_at, started_at, MINUTE) AS
ride_length_minutes,
    EXTRACT(DAYOFWEEK FROM started_at) AS day_number,
    EXTRACT(HOUR FROM started_at) AS hour_number,
    CASE
        WHEN start_station_name = end_station_name THEN 'Round Trip'
        ELSE 'One Way'
    END as trip_type
FROM `cyclistic-case-study-493121.CycleData.q1_2025_cleaned`
LIMIT 10;
```

Result/Observation: finalized a new dataset with, all previous data and new engineered data.

ANALYZE Repo

ANALYZE: Exploratory Data Analysis

The goal of the EDA session is to aggregate data on groups, identify counts and averages, and order data to determine trends amount different groups, i.e. times, type of riders, etc. The final produce resulting from this session a firm understanding of trends that can fuel marketing efforts of rental e-bikes.

Query Log:

- Query 25: Verify row count
- Query 26: Preview processed features
- Query 27: Compare total rides by user type
- Query 28: Compare rides by day of week and user type
- Query 29: Compare rides by hour of day and user type
- Query 30: Calculate average ride length by user type
- Query 31: Count rides over 30 minutes by user type
- Query 32: Compare rides by trip type and user type
- Query 33: Compare rides by bike type and user type
- Query 34: Rank top 10 start stations for casuals
- Query 35: Rank top 10 end stations for casuals

Query 25: Verify row count

None

SQL

```
SELECT COUNT(*) AS total_rows
FROM `cyclictic-case-study-493121.CycleData.q1_2025_processed`
```

Result/Observation: validating that all data was aggregated appropriately and no corruption occurred.

Query 26: Preview processed features

None

SQL

```
SELECT
    ride_id,
    member_casual,
    ride_length_minutes,
    day_number,
    hour_number,
```

```
trip_type
FROM `cyclistic-case-study-493121.CycleData.q1_2025_processed`
LIMIT 10;
```

Result/Observation: ensuring that all engineered data is properly loaded and is as meant to be.

Query 27: Compare total rides by user type

None

SQL

```
SELECT
    member_casual,
    COUNT(*) AS total_rides
FROM `cyclistic-case-study-493121.CycleData.q1_2025_processed`
GROUP BY member_casual;
```

Result/Observation: able to compare how many total rides are in each member and casual riders.

Query 28: Compare rides by day of week and user type

None

SQL

```
SELECT
    member_casual,
    day_number,
    COUNT(*) AS total_rides
FROM `cyclistic-case-study-493121.CycleData.q1_2025_processed`
GROUP BY member_casual, day_number
ORDER BY member_casual, day_number;
```

Result/Observation:

Identifying how members and casual riders differ based on the day the ride took place.

Query 29: Compare rides by hour of day and user type

None

SQL

```
SELECT
    member_casual,
    hour_number,
    COUNT(*) AS total_rides
FROM `cyclistic-case-study-493121.CycleData.q1_2025_processed`
GROUP BY member_casual, hour_number
ORDER BY member_casual, hour_number;
```

Result/Observation:

Identifying how member and casual riders compare as to the hour of the day.

Query 30: Calculate average ride length by user type

None

SQL

```
SELECT
    member_casual,
    ROUND(AVG(ride_length_minutes), 2) AS avg_ride_length
FROM `cyclistic-case-study-493121.CycleData.q1_2025_processed`
GROUP BY member_casual;
```

Result/Observation:

Identifying the average ride length in minutes of member riders vs. casual riders.

Query 31: Count rides over 30 minutes by user type

None

SQL

```
SELECT
    member_casual,
    COUNT(*) AS long_rides
FROM `cyclistic-case-study-493121.CycleData.q1_2025_processed`
```

```
WHERE ride_length_minutes >= 30
GROUP BY member_casual;
```

Result/Observation:

Identified how many riders in each group, members and casual riders, spend more than 30 rides riding per day.

Query 32: Compare rides by trip type and user type

None

SQL

```
SELECT
    member_casual,
    trip_type,
    COUNT(*) AS total_rides
FROM `cyclistic-case-study-493121.CycleData.q1_2025_processed`
GROUP BY member_casual, trip_type
ORDER BY member_casual, trip_type;
```

Result/Observation:

Identify the number of riders in distinct members and casuals by each trip type, to identify patterns between trip types and whether the riders are members or casual.

Query 33: Compare rides by bike type and user type

None

SQL

```
SELECT
    member_casual,
    rideable_type,
    COUNT(*) AS total_rides
FROM `cyclistic-case-study-493121.CycleData.q1_2025_processed`
GROUP BY member_casual, rideable_type
ORDER BY member_casual, rideable_type;
```

Result/Observation:

Identifying the number of riders in both members and casual riders, distinguished based on rideable type status.

Query 34: Rank top 10 start stations for casuals

None

SQL

```
SELECT start_station_name
FROM `cyclistic-case-study-493121.CycleData.q1_2025_processed`
WHERE member_casual = 'casual'
GROUP BY start_station_name
ORDER BY start_station_name
LIMIT 10;
```

Result/Observation:

Identified which start station has the most casual riders begin their ride with them.

Query 35: Rank top 10 end stations for casuals

None

SQL

```
SELECT end_station_name
FROM `cyclistic-case-study-493121.CycleData.q1_2025_processed`
WHERE member_casual = 'casual'
GROUP BY end_station_name
ORDER BY COUNT(*) DESC
LIMIT 10;
```

Result/Observation: Identify which end station has the most casual riders who end their ride with them.

End of Query Repository